

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЕТ ПО ЗАДАНИЮ №6

**«Сборка многомодульных программ.
Вычисление корней уравнений и определенных
интегралов.»**

Вариант 11 / 5 / 4

Выполнил:
студент 102 группы
Соловьев А. И.

Преподаватели:
Калинин Г. М.
Цыбров Е. Г.

Москва
2024

Содержание

Постановка задачи	2
Математическое обоснование	3
Минимальные условия сходимости	3
Результаты тестов	4
Структура программы и спецификация функций	9
Структура	9
Реализация	9
Сборка программы (Make-файл)	11
Программа на Си и на Ассемблере	13
Анализ допущенных ошибок	14
Список цитируемой литературы	15

Постановка задачи

Необходимо найти площадь фигуры, ограниченной данными тремя кривыми, лежащей на заданном отрезке. Гарантируется, что кривые попарно пересекаются на данном отрезке.

- Для нахождения точек пересечений требуется реализовать численный метод нахождения корней. Решение реализует два метода (на выбор): метод Ньютона (касательных) и метод хорд.
- Для нахождения площади фигуры требуется реализовать метод численного интегрирования. Решение реализует два метода (на выбор): метод трапеций и метод Симпсона (парабол).
- Отрезок для применения метода нахождения корней должен быть вычислен аналитически.

Функции, задающие кривые, и отрезок, на котором лежит фигура, задаются перед сборкой в файле. Функции задаются в обратной польской записи.

Пример:

```
1 0.0 4.0
2 2 x 4 / tan -
3 x
4 0.2 pi *
```

Математическое обоснование

Минимальные условия сходимости

- $\exists x_0 : f(x_0) = 0$
- $f(x)$ непрерывно дифференцируема на отрезке
- Начальная точка x достаточно близка к x_0

Для метода Ньютона желательно соблюдения все условия Фурье:

- $f'(x) \neq 0$ на отрезке
- $f''(x)$ не меняет знак на отрезке
- начальная точка x_0 с того конца отрезка, где $f(x_0) \cdot f''(x_0) > 0$,

так как при соблюдении лишь минимальных условий метод может уйти за пределы отрезка, сойтись к другому корню или не сойтись вовсе. [1]

Площадь заданной фигуры должна быть вычислена с точностью ε . В некоторой окрестности точки пересечения, которую мы ищем с точностью до ε_1 , функции эквивалентны своим касательным в этой точке. [1] Значит, при смещении границы интегрирования, изменение значения интеграла будет пропорционально квадрату смещения. Общая площадь фигуры равна комбинации трех интегралов с некоторыми знаками; общая погрешность может достигать $3\varepsilon_2 + 2\varepsilon_1^2 \leq 0.001$ по условию. Возьмем $\varepsilon_2 = \varepsilon_1 = 0.0001$, $3\varepsilon_2 + 2\varepsilon_1^2 = 0.0003 + 0.0000002 \approx 0.0003 \leq 0.001$.

Для вычисления интеграла с определенной точностью было использовано правило Рунге. Метод Симпсона имеет порядок точности 4, метод трапеций имеет порядок точности 15. Оценка погрешности равна разнице между двумя площадями, посчитанными с разной точностью, деленной на порядок точности. Поэтому количество разбиений в обоих методах экспоненциально повышается, пока оценка не станет меньше ε_2 . [1]

После нахождения точек, сначала определяется верхняя/нижняя ограничивающая кривая. Это кривая, которая не пересекает другие в средней из трех (по оси x) точке пересечения. Определив, лежит ли она ниже или выше этой точки, мы либо вычитаем ее интеграл из интегралов двух других функций, либо наоборот:

$$S = \text{sgn}(f_1(x_1) - f_2(x_1)) \cdot \left(\int_a^b f_1(x) dx - \int_a^{x_1} f_2(x) dx - \int_{x_1}^b f_3(x) dx \right).$$

Результаты тестов

Программа тестировалась на таких наборах данных:

1.

1	0.0 1.0
2	x
3	1 x -
4	x 0.5 - 2 ^

Аналитическое решение:

$$x^2 - x + 0.25 = x$$

$$D = (-2)^2 - 4 \cdot 0.25 = 3$$

$$x_{1,2} = \frac{2 \pm \sqrt{3}}{2} = 1 \pm \frac{\sqrt{3}}{2}, x_1 \in [0, 1]$$

$$x^2 - x + 0.25 = 1 - x, x^2 - 0.75 = 0$$

$$D = 4 \cdot 0.75 = 3$$

$$x_{1,2} = \frac{\pm \sqrt{3}}{2}, x_2 \in [0, 1]$$

$$x = 1 - x; x_3 = 0.5$$

$$\int x dx = \frac{1}{2}x^2 + C$$

$$\int -x + 1 dx = -\frac{1}{2}x^2 + x + C$$

$$\int (x - 0.5)^2 dx = \frac{1}{3}(x - 0.5)^3 + C$$

$$\frac{1}{2} \cdot (0.25 - (1 - \frac{\sqrt{3}}{2})^2) \approx 0.116$$

$$-\frac{1}{2} \cdot (\frac{3}{4} - 0.25) + (\frac{\sqrt{3}}{2} - 0.5) \approx 0.366 - 0.25 = 0.116$$

$$\frac{1}{3} \cdot (0.5 - \frac{\sqrt{3}}{2})^3 \approx 0.016$$

$$\frac{1}{3} \cdot (\frac{\sqrt{3}}{2} - 0.5)^3 \approx 0.016$$

$$S \approx 0.116 \cdot 2 - 0.016 \cdot 2 = 0.2$$

2.

```

1  0.0 1.0
2  x tan
3  1 x tan -
4  x tan 10 /

```

Аналитическое решение:

$$\tan(x) = 1 - \tan(x); \tan(x) = 0.5; x_1 = \arctan(0.5);$$

$$\tan(x) = \frac{\tan(x)}{10}; \tan(x) = 0; x_2 = 0;$$

$$1 - \tan(x) = \frac{\tan(x)}{10}; \tan(x) = \frac{10}{11}; x_3 = \arctan(\frac{10}{11})$$

$$\int \tan(x) dx = -\ln|\cos x| + C$$

$$\int 1 - \tan(x) dx = x + \ln|\cos x| + C$$

$$\int \frac{\tan(x)}{10} dx = \frac{-\ln|\cos x|}{10} + C$$

$$\begin{aligned}
S &= -\ln|\cos(\arctan(0.5))| + (\arctan(\frac{10}{11}) + \ln|\cos(\arctan(\frac{10}{11}))| - \\
&\quad - (\arctan(0.5) + \ln|\cos(\arctan(0.5))|) + \frac{\ln|\cos(\arctan(\frac{10}{11}))|}{10} = \\
&\approx 0.166
\end{aligned}$$

Результаты вычислений для двух вышеупомянутых тестовых примеров: координаты точек пересечения в таблицах и площадь полученной фигуры.

Кривые	x	y
1 и 2	0.5000	0.5000
2 и 3	0.8660	0.1339
1 и 3	0.1339	0.1339

Таблица 1: Координаты точек пересечения

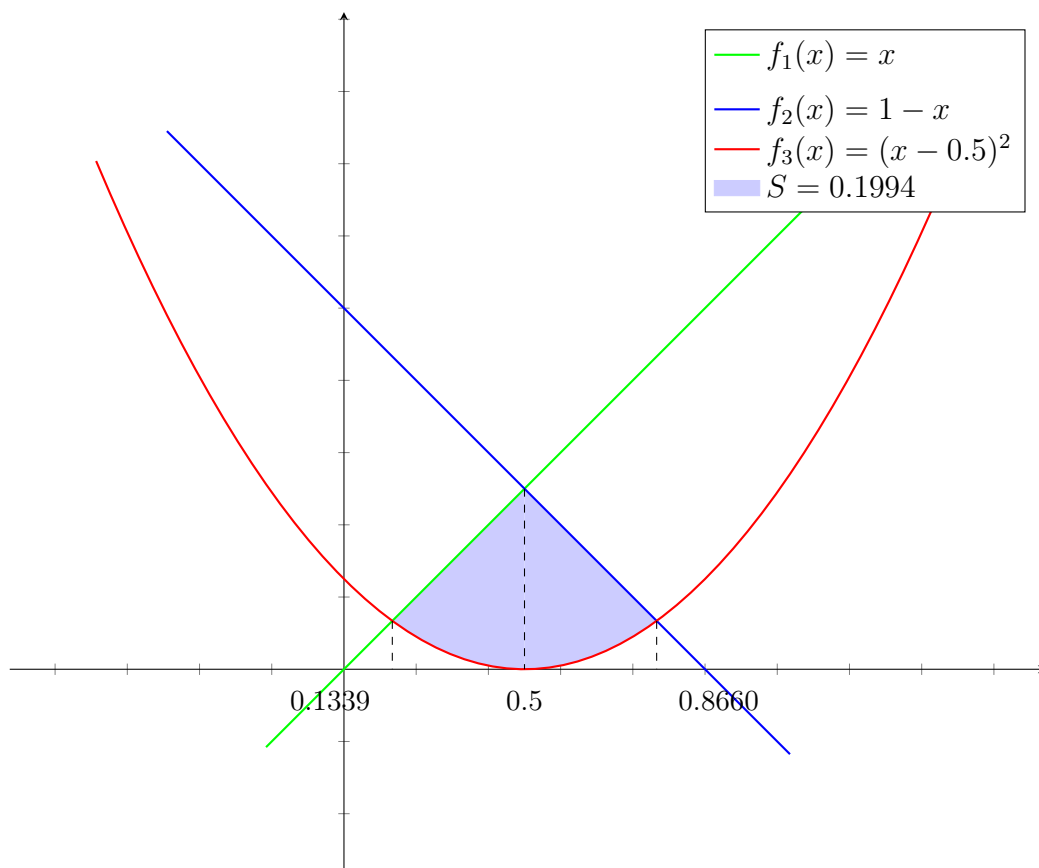


Рис. 1: Плоская фигура, ограниченная графиками заданных уравнений

Кривые	x	y
1 и 2	0.4636	0.5000
2 и 3	0.7378	0.091
1 и 3	0.0000	0.0000

Таблица 2: Координаты точек пересечения

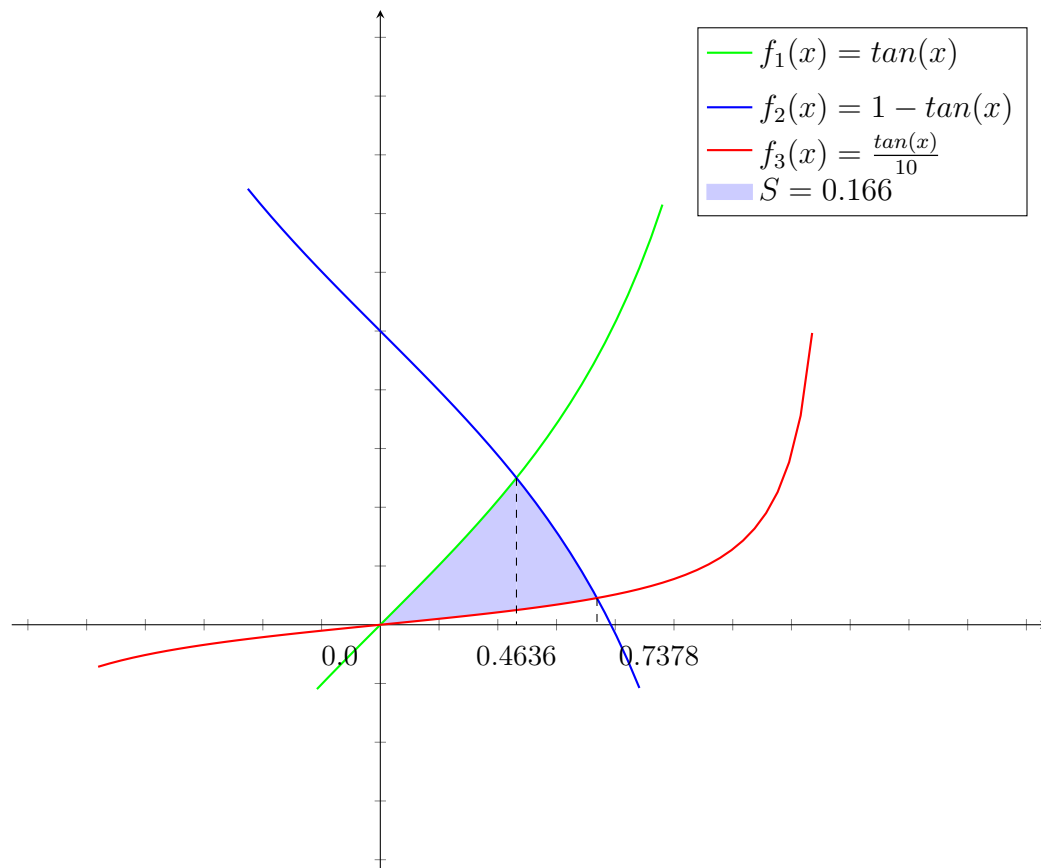


Рис. 2: Плоская фигура, ограниченная графиками заданных уравнений

Также, помимо аналитически решенных систем, были протестированы те, которые аналитически не решаются. Например:

```

1  0.0 4.0
2  2 x 4 / tan -
3  x
4  0.4 pi *
```

и

```

1  0.0 4.0
2  2 x 4 / tan -
3  x
4  0.6 pi *
```


Кривые	x	y
1 и 2	1.5823	1.5823
2 и 3 ₁	2.557	1.256
1 и 3 ₁	1.256	1.256
2 и 3 ₂	0.4581	1.885
1 и 3 ₂	1.885	1.885

Таблица 3: Координаты точек пересечения

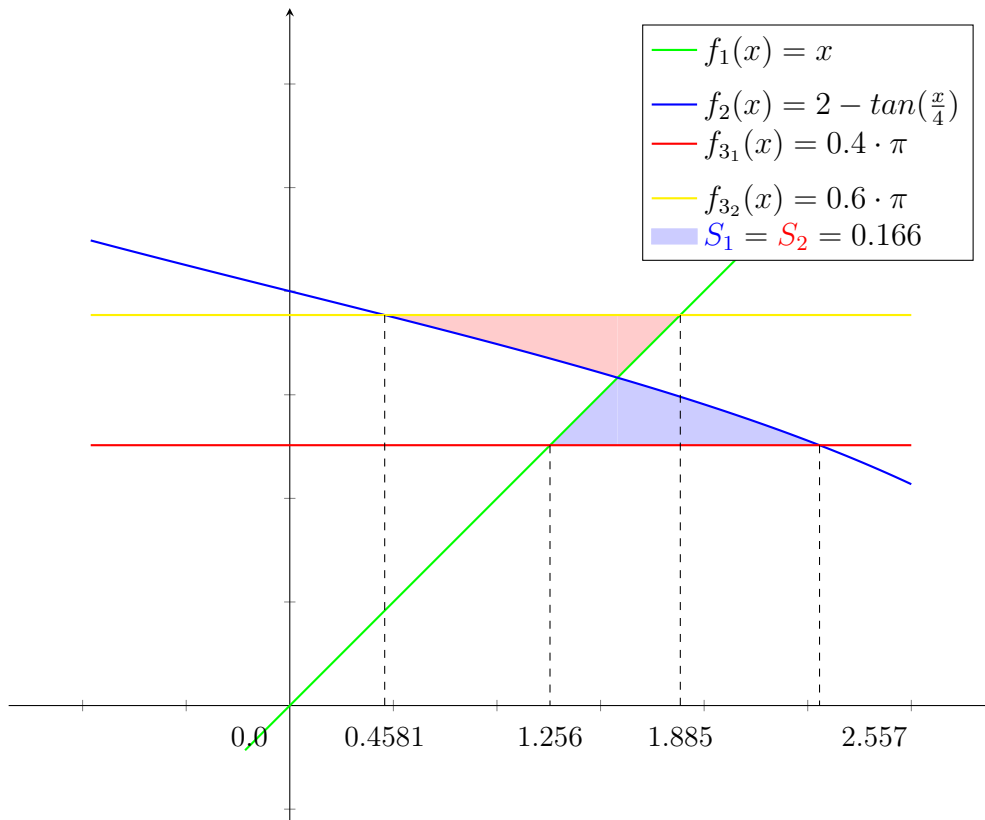


Рис. 3: Плоская фигура, ограниченная графиками заданных уравнений

Этот тест показывает, что площадь вычисляется одинаково независимо от положения фигуры в пространстве.

Структура программы и спецификация функций

Для ознакомления с кодом см. репозиторий [3] и документацию [2].

Структура

- `in.txt` – `SPEC_FILE`, содержащий отрезок и три функции
- `ast.c` и `diff_ast.c` – модули построения Abstract Syntax Tree и его дифференцирования.
- `rpn2asm.c` – строит дерево функций из `SPEC_FILE`, затем на основе дерева и дифференцированного дерева составляет ассемблерные функции в соответствии с `cdecl`.
- `funcs.asm` – ассемблерный модуль, составленный `rpn2asm.c`.
- `numeric.c` – модуль с реализацией численных методов нахождения корней и интегрирования.
- `main.c` – ввод и вычисления с применением функций из `numeric.c`.
- `Makefile` – сценарий автоматической сборки программы и отчета.
- `report.tex` – TeX файл с отчетом.

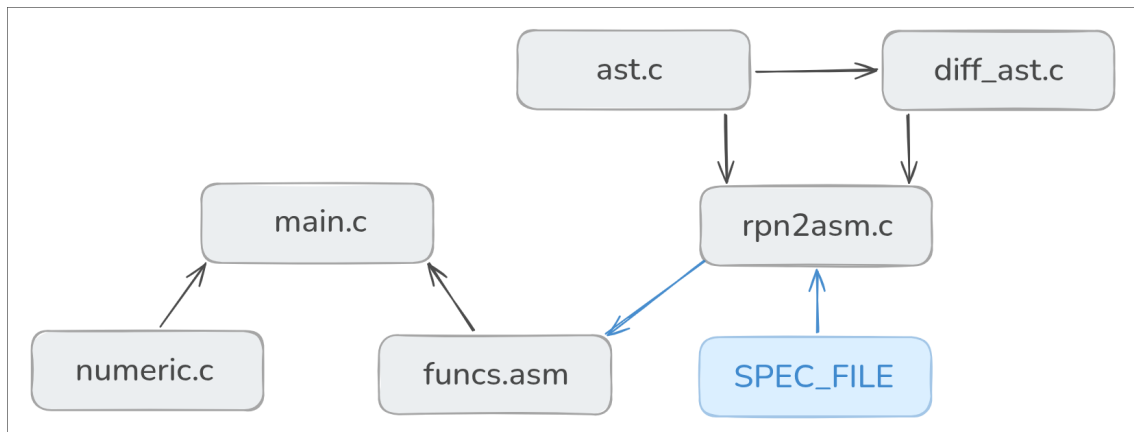


Рис. 4: Графическое представление связей компонентов

Реализация

Построение синтаксического дерева само по себе достаточно просто; но, в связи с тем, что в стеке FPU x87 всего 8 регистров, дерево требует оптимизации, чтобы его высота не превышала количество регистров. Оптимизации включают:

- Балансировку цепочек ассоциативных операций

- Удаление нулевых узлов и единичных множителей (появляется много после дифференцирования)
- Сокращение операций вида $\frac{f(x)}{f(x)} := 1$
- Предвычисление операций с константами (иногда появляются после дифференцирования)

Затем, на основе дерева строятся ассемблерные функции; дерево обходится в post-order порядке, левый операнд лежит в st1, правый в st0; после применения оператора кладем результат в st0.

Сборка программы (Make-файл)

Текст Makefile:

```
1 .PHONY: all force clean
2
3 SPEC_FILE?=in.txt
4 SAFE_EXP?=1
5 USE_NEWTON?=0
6 USE_SIMPSON?=1
7
8 CC := gcc
9 CFLAGS := -std=c99 -Wall -Wextra -m32 -O2 -DUSE_NEWTON=$(USE_NEWTON)
10 ↪ -DUSE_SIMPSON=$(USE_SIMPSON) -DSAFE_EXP=$(SAFE_EXP)
11 ↪ -DSPEC_FILE=\"$(SPEC_FILE)\" -DMADE_WITH_MAKE=1 -I include
12
13 all: report/report.pdf bin/main
14 force: ;
15 clean:
16     rm -rf obj
17     rm bin/gen
18     rm src/funcs.asm
19
20 bin/main: obj/main.o obj/numeric.o obj/funcs.o | bin
21     $(CC) -m32 -lm $^ -o $@
22     @if command -v figlet >/dev/null 2>&1; then figlet -f slant "main
23     ↪ built!"; else echo "main built\n\n\n"; fi
24
25 obj/funcs.o: src/funcs.asm
26     @mkdir -p $(dir $@)
27     nasm -f elf32 $< -o $@
28
29 obj/numeric.o: src/numeric.c force
30     @mkdir -p $(dir $@)
31     $(CC) $(CFLAGS) -c $< -o $@
32
33 obj/main.o: src/main.c force
34     @mkdir -p $(dir $@)
35     $(CC) $(CFLAGS) -c $< -o $@
36
37 src/funcs.asm: bin/gen $(SPEC_FILE)
38     tail -n +2 $(SPEC_FILE) | ./bin/gen
39
40 bin/gen: obj/rpn2asm.o obj/ast.o obj/diff_ast.o | bin
41     $(CC) -m32 -lm $^ -o $@
42
43 obj/%.o: src/%.c
44     @mkdir -p $(dir $@)
45     $(CC) $(CFLAGS) -c $< -o $@
```

```

44 report/report.pdf: report/report.tex report/graph.png
   ↪ report/make_graph.png
45     pdflatex -shell-escape -output-directory=report report/report.tex
46     @rm report/report.aux
47     @rm report/report.log
48     @rm report/report.toc
49
50     @if command -v figlet >/dev/null 2>&1; then figlet -f slant
   ↪ "report built!"; else echo "report built\n\n\n"; fi
51 bin:
52     @mkdir -p bin

```

Поскольку изменение флагов (переменных окружения) `USE_X` make не считает изменением зависимостей, для удобного изменения используемых методов файлы `numeric.o` и `main.o`, зависящие от этих флагов, пересобираются каждый раз. Для этого им задана зависимость `force`, обновляемая каждый раз.

Переменные окружения (в скобках – значение по умолчанию):

- `USE_NEWTON=0,1 (0)` – использовать метод Ньютона (касательных) или метод хорд
- `USE_SIMPSON=0,1 (1)` – использовать метод Симпсона (парабол) или метод трапеций
- `SPEC_FILE=<filename> (in.txt)` – файл с отрезком и кривыми
- `SAFE_EXP=0,1 (1)` – использовать упрощенную экспоненту (см. стр. 14)

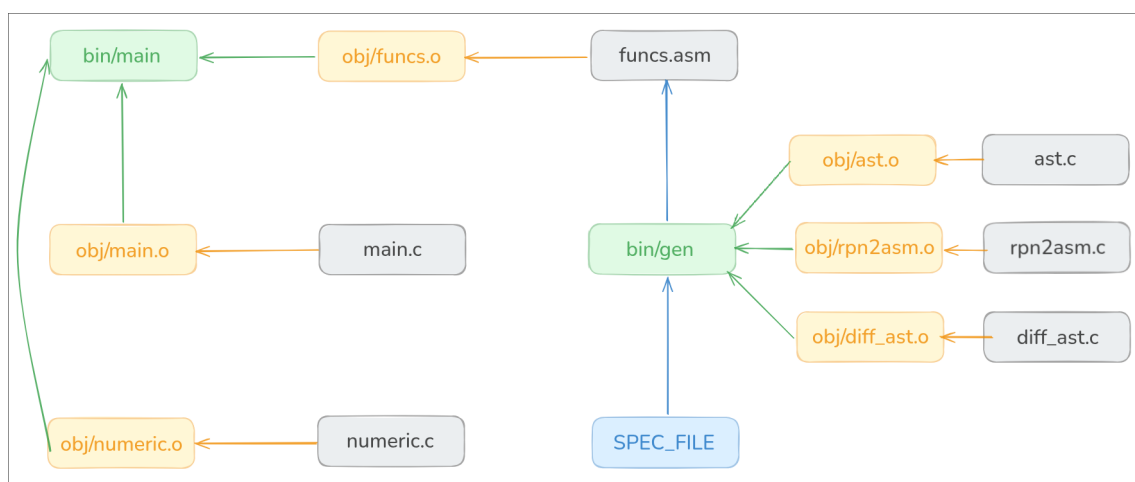


Рис. 5: Графическое представление зависимостей

Анализ допущенных ошибок

Задание не включало реализацию оператора $^$; он был реализован, но с нюансами. В FPU x87 нет прямой инструкции для возведения a в степень b ; есть инструкция `fyl2x`, вычисляющая $a \cdot \log_2(b)$, и инструкция `f2xm1`, вычисляющая $2^x - 1$ для $x \in [-1, 1]$; таким образом, $a^b = 2^{b \cdot \log_2(a)} = 2^{\text{round}(b \cdot \log_2(a))} \cdot 2^{b \cdot \log_2(a) \% 1}$ (целая и дробная части).

Проблемы возникают, когда $a \leq 0$; $\log_2(0) = -\infty$, для отрицательных – *nan*. Отрицательный аргумент появляется очень часто, когда вычисляется, например, значение $(x - a)^2$ при $x < a$; поэтому в программе полноценно реализовано возведение только в константную степень, которая превращается в обычное перемножение в ходе построения дерева. В Makefile для включения реальной экспоненты нужно установить `SAFE_EXP=0`.

Список литературы

- [1] Ильин В. А., Садовничий В. А., Сендов Бл. Х. Математический анализ. Т. 1 — Москва: Наука, 1985.
- [2] <https://artteam8.github.io/1st-Year-Final-Boss>
- [3] <https://github.com/artteam8/1st-Year-Final-Boss>